Trabalho 1 – Concorrência e Sincronização: O Problema dos Filósofos Jantando

Disciplina: FPPD · **Grupo:** Henrique Horch, Felipe Araujo, Eduardo Costa · **Repositório:** https://github.com/eduardocsilveira/FPPD_T1

1. Soluções e mecanismos de sincronização

Fizemos as quatro versões em Go, uma por arquivo (`v0/main.go` a `v3/main.go`). Os parâmetros são os mesmos em todas: 5 filósofos, 20 ciclos, 50 ms para pensar e 50 ms para comer. O que muda de uma versão para outra é só o jeito de pegar os garfos. O filósofo *i* tem o garfo *i* à esquerda e o garfo (*i*+1) mod 5 à direita. A V0 parte do `filo_seq.go` do Moodle; aumentamos para 20 ciclos e baixamos os tempos de 2 s e 3 s para 50 ms, senão cada execução da V0 levaria mais de 8 minutos.

Versão	Como funciona	Mecanismo em Go
V0	Dois laços, um dentro do outro: cada filósofo faz seus 20 ciclos de pensar e comer e só depois o próximo começa.	Nenhum. Sem goroutines não tem disputa por garfo.
V1	Uma goroutine por filósofo. Cada um pega o garfo esquerdo, espera 5 ms e pega o direito. O programa imprime quem pegou e quem está esperando.	Um `sync.Mutex` por garfo: `Lock` pega, `Unlock` solta. O `main` espera as goroutines com `sync.WaitGroup`.
V2	Igual à V1, só que antes de pegar os garfos o filósofo trava a mesa inteira, e só destrava depois de soltar os dois.	Mais um `sync.Mutex`, global, chamado `mesa`. Na prática é um semáforo binário em volta da refeição toda.
V3	Sem a trava da mesa. Cada filósofo pega primeiro o garfo de número menor. Só muda para o filósofo 4, que passa a pegar o garfo 0 antes do 4.	Um `sync.Mutex` por garfo, sempre travados em ordem crescente de número.

Por que a V1 trava. Os cinco terminam de pensar quase ao mesmo tempo e cada um pega o garfo da esquerda. Quando vão pegar o da direita, ele já está com o vizinho, e aí ninguém anda mais. As quatro condições de deadlock aparecem juntas: **exclusão mútua** (um mutex tem um dono só), **segura e espera** (o filósofo não larga o garfo esquerdo enquanto espera o direito), **ausência de preempção** (só quem deu `Lock` dá `Unlock`, ninguém toma o garfo de ninguém) e **espera circular** (F0 espera F1, que espera F2, até F4, que espera F0).

Por que V2 e V3 não travam. Na V2 só um filósofo de cada vez chega a tentar pegar garfo, então os dois garfos dele estão sempre livres e o segura e espera deixa de existir. A V3 ataca outra condição, a espera circular. Como todo mundo pega o garfo de menor número antes, quem está esperando um garfo só pode estar segurando um de número menor; seguindo essa cadeia os números só sobem e ela nunca volta para o começo.

2. Análise comparativa

A V0 levou 10,19 s em média, que é o que se espera de 100 ciclos de 100 ms em fila. A V2 fez em 5,15 s, 1,98 vezes mais rápido. Esse ganho vem só do pensar: enquanto um filósofo come, os outros já estão pensando. As refeições continuam uma de cada vez (100 × 50 ms = 5 s). Já a V3 terminou em 2,74 s, 3,71 vezes mais rápida que a V0 e 1,88 vezes mais rápida que a V2. Com cinco garfos dá para no máximo dois comerem juntos, então o melhor tempo possível para as 100 refeições seria 2,5 s. A V3 ficou 10% acima disso.

Onde a V2 mais perde é na espera. Cada filósofo ficou em média 150,0 ms parado por ciclo, contra 24,8 ms na V3, umas 6,1 vezes mais. Dá para chegar nesse número na mão: são cinco filósofos para uma vaga só, então cada um come uma vez a cada 250 ms; tirando os 50 ms pensando e os 50 ms comendo, sobram 150 ms de fila. A maior espera que medimos foi 204,7 ms na V2 e 153,1 ms na V3. Na V2 nunca houve mais de 1 filósofo comendo; na V3 chegaram a 2. Na V0 não existe espera, porque só há um filósofo ativo por vez. Nas três versões que terminam, todos fecharam os 20 ciclos.

A V1 ficou de fora da tabela porque não termina. Travou em 5 das 5 execuções, logo depois do primeiro pensar, sem nenhum ciclo concluído. O código não garante o deadlock: ele depende de os cinco pegarem o garfo esquerdo antes de alguém chegar no direito. Só que todos terminam de pensar praticamente juntos, e é isso que acontece na prática.

3. Resultados experimentais

Médias de 5 execuções por versão. Tempos em milissegundos. A maior espera é o máximo entre todas as execuções.

Versão	Sincronização	Tempo total (média)	Tempo total (mín – máx)	Espera média	Maior espera	Máx. comendo juntos	Ciclos por filósofo
V0	nenhuma (sequencial)	10.185,5	10.182,1 – 10.188,8	0	0	1	20 cada
V2	trava única da mesa	5.146,0	5.140,6 – 5.151,4	149,99	204,74	1	20 cada
V3	ordenação dos garfos	2.741,8	2.702,5 – 2.752,6	24,77	153,11	2	20 cada
V1	mutex por garfo, esq. → dir.					deadlock em 5 de 5 execuções; 0 ciclos concluídos	

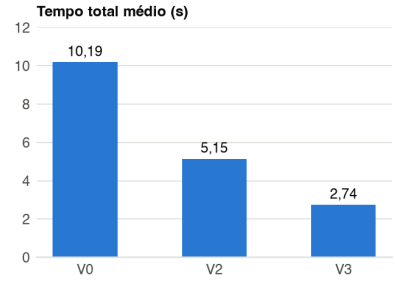
Na V0 não medimos espera nem simultaneidade: o programa é sequencial, então a espera é zero e só um come por vez.

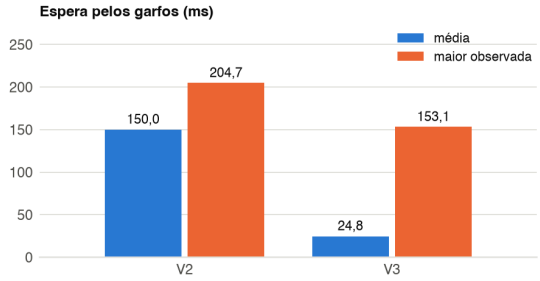
V2, execução por execução

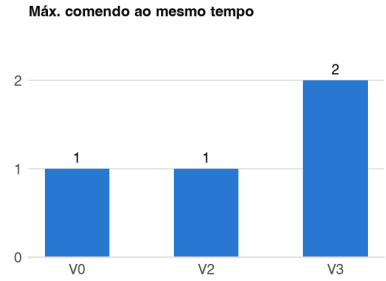
Exec.	Tempo total	Espera média	Maior espera	Máx. juntos
1	5.145,6	150,28	203,74	1
2	5.140,6	150,15	204,13	1
3	5.148,0	149,32	204,74	1
4	5.151,4	149,95	204,50	1
5	5.144,5	150,24	203,88	1

V3, execução por execução

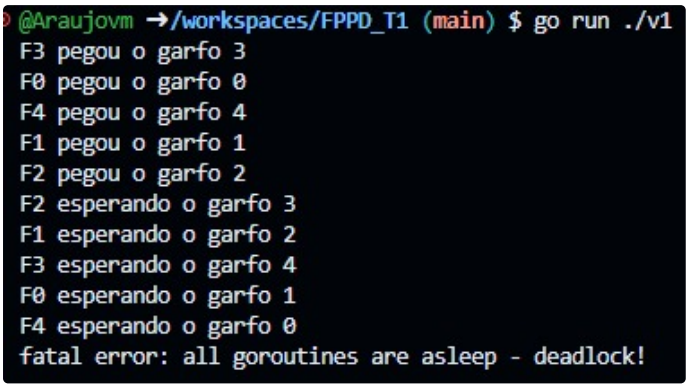
Exec.	Tempo total	Espera média	Maior espera	Máx. juntos
1	2.752,6	26,50	152,64	2
2	2.752,3	23,45	102,57	2
3	2.751,6	24,46	153,11	2
4	2.749,9	25,47	102,59	2
5	2.702,5	23,98	102,44	2



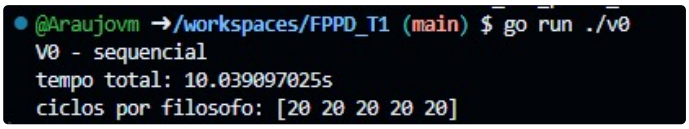


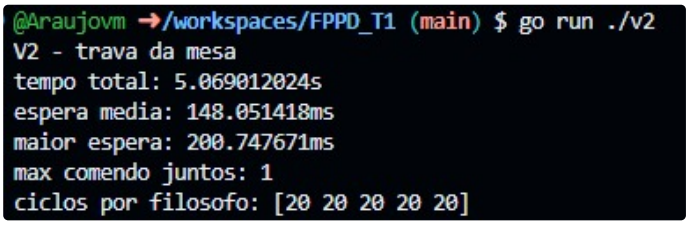


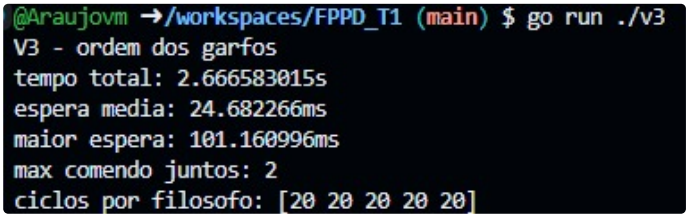
Capturas de tela das execuções



V1: evidência do deadlock







Os prints foram tirados no GitHub Codespaces (Linux), rodando o código do repositório. Já a tabela e os gráficos foram medidos no Apple M1 da seção 5, por isso os tempos dos prints não batem exatamente com os da tabela. No print da V1, as linhas "pegou" mostram o garfo que cada filósofo tem e as linhas "esperando" mostram o garfo que falta; como nenhuma goroutine consegue continuar, o Go encerra o programa com `fatal error`. As saídas completas das execuções medidas estão em `resultados/saidas/`.

4. Conclusão e respostas

1. Como funciona cada solução?

A V0 roda os filósofos em fila, um depois do outro, sem concorrência. Na V1 cada filósofo vira uma goroutine que pega o garfo esquerdo e depois o direito. A V2 faz a mesma coisa, mas dentro de uma trava que vale para a mesa inteira. A V3 não tem essa trava; nela todos pegam os garfos em ordem crescente de número.

2. Quais mecanismos de sincronização são utilizados?

Exclusão mútua com um mutex por garfo (V1, V2 e V3), uma trava global na V2 e ordenação dos recursos na V3. A V0 não precisa de nenhum.

3. Como esses mecanismos são implementados em Go?

Cada garfo é um `sync.Mutex`, guardado em um vetor `[5]sync.Mutex`. `Lock` bloqueia a goroutine até o garfo ficar livre e `Unlock` devolve. A mesa da V2 é só mais um `sync.Mutex`. Os filósofos são goroutines, criadas com `go`, e o `main` espera por elas com `sync.WaitGroup`. Para a ordenação da V3 não usamos nada novo: comparamos os dois números e travamos o menor primeiro. As métricas ficam em variáveis globais com um mutex só delas (`mu`), que não entra na disputa pelos garfos.

4. Por que ocorre deadlock na V1?

Porque dá tempo de os cinco pegarem o garfo esquerdo antes de qualquer um pegar o direito. O print da página 2 mostra isso: F0 está com o garfo 0 e espera o 1, F1 está com o 1 e espera o 2, F2 com o 2 espera o 3, F3 com o 3 espera o 4, e F4 está com o 4 esperando o 0. O garfo que cada um quer está com o vizinho, que também está parado, e a volta se fecha no F0. Com as quatro condições presentes ao mesmo tempo, o programa travou em 5 das 5 execuções e o Go encerrou com `fatal error: all goroutines are asleep – deadlock!`.

5. Como V2 e V3 evitam deadlock?

A V2 acaba com o segura e espera: quem vai pegar garfos já está com a mesa, e nessa hora os dois garfos estão livres. A V3 acaba com a espera circular. Como o filósofo 4 agora disputa o garfo 0 com o filósofo 0 antes de segurar qualquer coisa, nunca ficam os cinco com um garfo na mão. As duas terminaram os 100 ciclos em todas as execuções. Também rodamos com o detector de corrida (`go run –race`) e não apareceu nenhum aviso.

6. Existe possibilidade de starvation?

Na V1 travada existe, e para todos, já que ninguém volta a comer. Na V2 e na V3 o algoritmo não garante quem é atendido primeiro, porque o `sync.Mutex` não funciona como uma fila rígida. Em teoria um filósofo poderia ficar para trás se os outros passassem sempre na frente dele. Nos nossos testes não aconteceu: todos fecharam os 20 ciclos em todas as execuções, e a maior espera foi de 205 ms na V2 (umas quatro refeições dos outros) e 153 ms na V3 (cerca de 3 refeições de vizinhos). Então é possível em teoria, mas não observamos.

7. Qual solução apresentou maior tempo total de execução?

A V0, com 10.185 ms em média. Entre as concorrentes que terminam, a V2: 5.146 ms contra 2.742 ms da V3.

8. Qual solução apresentou maior tempo de espera pelos recursos?

A V2, com 150,0 ms de espera média e 204,7 ms de maior espera. Na V3 foram 24,8 ms e 153,1 ms. Na V0 não há espera, porque nunca tem outro filósofo ativo.

9. Quantos filósofos conseguiram comer simultaneamente em cada solução?

Na V0, 1. Na V2, 1. Na V3, 2, que é o máximo possível com cinco garfos. Na V1 nenhum chegou a comer, porque o programa travou antes da primeira refeição.

10. Qual foi o impacto da sincronização excessiva da V2?

Com a trava da mesa as refeições viraram uma fila única. O tempo total foi de 5.146 ms, quase os 5.000 ms das 100 refeições em série, e 1,88 vezes o tempo da V3. Cada filósofo esperou em média 150 ms por ciclo, o triplo do tempo que passa comendo. As cinco execuções ficaram entre 5.141 e 5.151 ms. Variou tão pouco que esse custo é da própria solução, não do escalonamento.

11. A solução com maior sincronização foi necessariamente a de melhor desempenho?

Não. A V2 é a que mais restringe os filósofos e foi a pior das versões concorrentes em tudo que medimos: levou 1,88 vezes o tempo da V3, teve 6,1 vezes a espera média e metade dos filósofos comendo juntos. As duas estão corretas. A diferença é que a V3 proíbe só o que precisa para não fechar o ciclo de espera, e a V2 proíbe também coisas que seriam seguras, como dois filósofos que não são vizinhos comerem ao mesmo tempo.

12. Qual é a relação entre o grau de concorrência observado e o tempo total de execução?

Quanto mais coisa acontece ao mesmo tempo, menor o tempo total. Na V0 nada se sobrepõe e foram 10,2 s. Na V2 só o pensar se sobrepõe (um comendo por vez) e caiu para 5,1 s. Na V3 as refeições também se sobrepõem (dois comendo) e foi para 2,7 s. Ou seja, dobrar o número de filósofos comendo juntos cortou o tempo quase pela metade. Um detalhe: V0 e V2 têm o mesmo máximo de filósofos comendo (1) e tempos bem diferentes, então esse número sozinho não mede a concorrência. O pensar ao mesmo tempo também conta.

5. Ambiente de execução

Linguagem e versão	Go 1.27.0 (darwin/arm64)
Sistema operacional	macOS 13.4 (Darwin 22.5.0)
Processador	Apple M1
Cores disponíveis	8
Compilação e execução	`go run ./v0`, `go run ./v1`, `go run ./v2`, `go run ./v3` (o `go run` compila e executa)
Capturas de tela	GitHub Codespaces (Linux), com o mesmo código do repositório. As medições da tabela não foram feitas nele.
Experimento completo	`scripts/rodar.sh` roda 5 vezes cada versão e salva as saídas em `resultados/saidas/`.
Código-fonte	https://github.com/eduardocsilveira/FPPD_T1